Skills vs Plugins vs CLIs vs MCP Servers

Comprehensive Comparison | 2026 Research Corpus | NexusPoint

The One-Line Definitions

	What it is
Skill	An instruction package that tells Claude how to behave on a specific task
Plugin	A distribution bundle that ships skills + MCP servers + hooks + subagents together
CLI	A shell-executable tool that operates outside Claude's context window
MCP Server	A running process that gives Claude real-time access to external tools and data

How Each One Works

Skills

A **SKILL.md** file in **.claude/skills/<name>/**. Never executes - pure instructions loaded into Claude's context when a matching task is detected. [s33, s120]

Progressive disclosure: name + description (~100 tokens) loads at session start. Full body (<5K tokens) only loads when the task matches the trigger. Keeps context lean.

Two kinds:

- **Capability uplift** - new actions Claude can take ("follow this TDD workflow", "run this research pipeline")
- **Encoded preference** - your style rules baked in ("never use em dashes", "always cite sources")

Plugins

The **packaging layer** - bundles one or more of: skills, MCP server configs, hooks, and subagent definitions into a single installable unit. Install via **/plugin install <name>** or the marketplace. [s58]

Think of it as a npm package for Claude Code config. The plugin author packages primitives; you install the package. Superpowers, for example, ships ~15 skills + hook rules + subagent templates as one install.

CLIs

A CLI runs in your shell as a **standalone process** - completely outside Claude's context window. Doesn't talk to Claude unless explicitly wired to call the Claude API.

Two subcategories:

- **Claude Code itself** - the primary CLI (claude). The agentic runtime.
- **Companion CLIs** - third-party tools that enhance the experience: Claudebase syncs config, claude-mem manages SQLite memory, getburnd audits token costs, gh powers git workflows.

MCP Servers

A running server process (local or remote) implementing the **Model Context Protocol** - an open JSON-RPC standard. Claude connects to it and gains access to the tools/resources it exposes. No custom glue code. [s172, s213]

When connected, its tools appear in Claude's context as callable functions - like giving Claude a hand that can reach into GitHub, Postgres, Slack, Firecrawl, etc. in real time.

Side-by-Side Comparison

Dimension	Skill	Plugin	CLI	MCP Server
What it adds	Behavioral instruction	Bundled config (skills+hooks+MCP)	OS-layer tool	Live external data/action
Format	SKILL.md markdown	Installable package	Executable binary/script	JSON-RPC server process
Runs where	Inside Claude's context	N/A (it's a bundle)	Shell, outside Claude	Separate process
Executes code?	No - pure instruction	No (bundles things that might)	Yes - runs independently	Yes - serves tool calls
Persists sessions?	Yes (file stays)	Yes (installed config)	Yes (binary stays)	Yes (server config stays)
Authors	You / community	Community / Anthropic	You / open-source	Community / vendors / you
Scope of impact	Claude behavior on a task	All primitives in one install	Outside Claude entirely	Claude's reach externally
Token cost	~100 at startup, <5K on trigger	Depends on what's bundled	Zero - outside context	Low - tool call overhead only
Discovery	.claude/skills/ or plugins	awesome-claude-code / marketplace	npm/pip/GitHub	awesome-mcp-servers
Best analogy	A playbook or SOP	An app installer	A command-line app	A REST API adapter

How They Relate to Each Other

Plugin
+-- ships **Skills** - loaded into Claude's context as instructions
+-- ships **Hooks** - event scripts (PreToolUse / PostToolUse / SessionStart)
+-- ships **MCP configs** - wires Claude to external tool servers
+-- ships **Subagents** - isolated-context agents for parallel work

CLI (companion)
+-- operates in the shell layer - can call Claude API, sync config, audit costs

MCP Server
+-- runs as a process - Claude calls its tools at runtime (GitHub, Slack, Firecrawl, etc.)

A **Plugin** is the container. **Skills** are the brains inside it. **MCP Servers** are the hands that reach outside. **CLIs** are the tools you use in the terminal that exist entirely outside this stack.

When to Reach for Each

Need	Use
Encode a repeatable workflow (TDD, debugging, code review)	Skill
Install a pre-built productivity boost (Superpowers, Frontend Design)	Plugin
Give Claude real-time access to GitHub / Slack / Notion / Postgres	MCP Server
Sync config across machines, audit token costs, manage memory	CLI (companion)
Build your own Claude-powered tool for internal/client use	CLI via Agent SDK
Add hook behavior without shipping a full plugin	Skill with hooks (or settings.json directly)

The Nuance Most People Miss

- **Skills and plugins don't execute.** They influence Claude's instructions. Execution happens in Claude's agentic loop or via MCP/CLI calls.

- **A plugin without MCP servers is just a skill bundle.** The magic of Superpowers is that it codifies the entire SDLC workflow as skills + hooks in one install - not new code execution.

- **MCP is the only primitive that crosses the system boundary.** Skills, plugins, and most CLIs are local config. MCP servers connect Claude to the real world in real time.

- **Context cost matters.** A skill adds ~100 tokens at startup (full body only on trigger). A poorly configured MCP server that dumps large responses can tank your session. CLIs add zero tokens to context.

Source: Claude Advisor skill - 2026 NotebookLM research corpus (237 sources) [s10, s29, s32, s33, s58, s69, s120, s150, s172, s213]. NexusPoint internal reference.